

JONES DAO · SECURITY REVIEW

Jones DAO

jGM.

A security review of jGM.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Jones DAO	jGM	Guardian	April 2 – April 24, 2024	May 1, 2024

§ 00

Contents.

107 pages — overview, confidence ranking, scope, 77 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 48 findings	P. 13
	Remediation Review · 29 findings	P. 74
§ 05	Disclaimer	P. 106
§ 06	About Guardian	P. 107

§ 01

Overview.

Guardian reviewed jGM across 2 review rounds. The final report includes 19 Critical, 10 High, 24 Medium, 24 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	Jones-DA0/jones-gm
CLIENT	METHODOLOGY
Jones DAO	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
jGM	8e661971ca0a
PREPARED BY	REMEDATION REVIEW COMMIT
Guardian	c40df2177c9c
AUDIT WINDOW	
April 2 – April 24, 2024	
FINAL REPORT	
May 1, 2024	

1.1 Confidence Ranking

1 / 5

Very Low Confidence

Guardian assigns a Confidence Ranking of 1 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	19	0	4	7	8
● High	10	0	5	4	1
● Medium	24	0	13	5	6
● Low	24	0	9	0	15
● Informational	0	0	0	0	0
All severities	77	0	31	16	30

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence ASSIGNED	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed jGM in `Jones-DA0/jones-gm`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Leverage Strategy DoS Through Depositing And Withdrawing Dust	DoS	CRITICAL	PARTIALLY RESOLVED	p. 13
C-02	Leverage Strategy Debt Pay Back Fails Due To Insufficient Allowance	Logical Error	CRITICAL	RESOLVED	p. 15
C-03	Attacker Can DOS GM Strategy By Abusing Blackisted USDC Addresses	DoS	CRITICAL	RESOLVED	p. 17
C-04	Function isInRange will not catch out of range values	Logical Error	CRITICAL	RESOLVED	p. 18
C-05	Value Extracted During Faulty Rebalance	Logical Error	CRITICAL	RESOLVED	p. 19
C-06	Borrowed USDC Is Counted As User Deposit	Logical Error	CRITICAL	RESOLVED	p. 20
C-07	Full Protocol DOS Via Spamming Operations	DoS	CRITICAL	PARTIALLY RESOLVED	p. 21
C-08	Attacker Can Repurpose Signatures To Damage Protocol	Logical Error	CRITICAL	RESOLVED	p. 22
C-09	Attacker can cause DoS by setting receiver to address(0)	DoS	CRITICAL	RESOLVED	p. 24
C-10	Liquidated Users Can Steal Future Deposits	Logical Error	CRITICAL	PARTIALLY RESOLVED	p. 25
C-11	Attacker can call cancellation callback to deplete strategy of GM tokens	Validation	CRITICAL	PARTIALLY RESOLVED	p. 27

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
C-12	Attacker Can cause DoS by Intentionally exceeding range check	DoS	CRITICAL	PARTIALLY RESOLVED	p. 28
C-13	Compound and Rebalance do not increase the nonce	Logical Error	CRITICAL	RESOLVED	p. 30
C-14	Attacker can deposit while ongoingOperation is true to mint extra shares	Logical Error	CRITICAL	PARTIALLY RESOLVED	p. 31
C-15	Attacker can cause DoS by overriding gmxData key	DoS	CRITICAL	PARTIALLY RESOLVED	p. 33
H-01	GM Strategy Token Weight Is Errantly Determined	Logical Error	HIGH	PARTIALLY RESOLVED	p. 35
H-02	Withdraws Over Leverages The System	Logical Error	HIGH	ACKNOWLEDGED	p. 36
H-03	Incorrect Underlying jGM Calculations Due To Fees And Slippage	Logical Error	HIGH	PARTIALLY RESOLVED	p. 37
H-04	Depositing After Unwind Reverts	Validation	HIGH	RESOLVED	p. 39
H-05	Execution features may be disabled	Validation	HIGH	PARTIALLY RESOLVED	p. 40
H-06	User can always use stale price	Validation	HIGH	PARTIALLY RESOLVED	p. 41
M-01	Manipulated swap pool will cause DoS on withdrawals	DoS	MEDIUM	RESOLVED	p. 42
M-02	GMRouter Deposits And Withdraws Are Not EIP712 Compliant	Logical Error	MEDIUM	PARTIALLY RESOLVED	p. 43
M-03	Potential Tokens Trapped On GM Strategy Deposits	Validation	MEDIUM	PARTIALLY RESOLVED	p. 44
M-04	Executing Partial Or Premature Pending Operations Blocks Further Rebalances	Logical Error	MEDIUM	RESOLVED	p. 46

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-05	Trapped ETH On keeperPayBack Call	Validation	MEDIUM	PARTIALLY RESOLVED	p. 48
M-06	Leverage Down On Zero Underlying Leaves Action Uncleared	Logical Error	MEDIUM	RESOLVED	p. 49
M-07	New Leverage Incorrectly Calculated During Leverage Up	Logical Error	MEDIUM	RESOLVED	p. 50
M-08	Value Extraction By Withdrawing After A Pay Back	Logical Error	MEDIUM	PARTIALLY RESOLVED	p. 52
M-09	Admin cannot change operationOngoing when stuck	Logical Error	MEDIUM	RESOLVED	p. 54
M-10	Jones does not collect from jonesRate when protocolRate is zero	Logical Error	MEDIUM	RESOLVED	p. 55
M-11	Users cannot borrow when stable vault is low on borrowable funds	Logical Error	MEDIUM	PARTIALLY RESOLVED	p. 56
M-12	Deterministic keeper actions can be gamed	Logical Error	MEDIUM	ACKNOWLEDGED	p. 57
L-01	Strategy will payback users underlying when there is insufficient debt	Logical Error	LOW	RESOLVED	p. 58
L-02	Misconfigured handler addresses will lead to failed callbacks	Validation	LOW	RESOLVED	p. 59
L-03	Missing check if action does not equal 1 in Keeper PayBack	Validation	LOW	RESOLVED	p. 60
L-04	Swapper Incorrect Allowance Handling	Logical Error	LOW	RESOLVED	p. 61
L-05	Protocol Withdraw Incentives Cannot Be Changed	Logical Error	LOW	RESOLVED	p. 62
L-06	Keeper Pay Back Functionality Overwrites Current Action	Logical Error	LOW	RESOLVED	p. 63
L-07	Fee Rates Can Dos Protocol	Logical Error	LOW	RESOLVED	p. 64

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-08	Deprecated Chainlink Staleness Check	Logical Error	LOW	RESOLVED	p. 65
L-09	Possible Trapped ETH In Routers	Logical Error	LOW	RESOLVED	p. 66
L-10	Leverage Down Reverts On Zero Underlying	Logical Error	LOW	RESOLVED	p. 67
L-11	Allowances Should Be Cleared While Updating GMX Variables And Swappers	Logical Error	LOW	RESOLVED	p. 68
L-12	Leverage Strategy Should Not Be Updated During An Operation	Logical Error	LOW	RESOLVED	p. 69
L-13	Rebalance Purchases Can Be Done In Multiple Stages	Validation	LOW	RESOLVED	p. 70
L-14	Incentive Not Accounted For on Payback Stables Availability	Validation	LOW	RESOLVED	p. 71
L-15	Strategy Total Value Does Not Reflect Rebalancing Amounts	Logical Error	LOW	RESOLVED	p. 73

REMEDIATION REVIEW

C-01	User Withdrawals Are Accounted As Stable Debt Payment After A Failed Swap	Logical Error	CRITICAL	ACKNOWLEDGED	p. 74
C-02	Attacker Can Repurpose Signatures To Damage Protocol	Logical Error	CRITICAL	ACKNOWLEDGED	p. 76
C-02.2	Using L1 block number leads to DoS	Logical Error	CRITICAL	ACKNOWLEDGED	p. 77
C-03	USDC Withdraw Amount Not Updated After Protocol Trim	Logical Error	CRITICAL	ACKNOWLEDGED	p. 78
H-01	GM Strategy Token Weight Is Errantly Determined	Logical Error	HIGH	ACKNOWLEDGED	p. 79

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
H-01.2	Data can still be stale and impact the share price calculation	Logical Error	HIGH	ACKNOWLEDGED	p. 80
H-02	Wrong key used to check if execution is disabled.	DoS	HIGH	ACKNOWLEDGED	p. 82
H-03	Attacker can donate USDC to halt rebalance.	DoS	HIGH	ACKNOWLEDGED	p. 83
M-01	Incorrect Underlying jGM Calculations Due To Fees	Logical Error	MEDIUM	ACKNOWLEDGED	p. 84
M-01.2	Admin cannot change rebalanceLock when stuck	DoS	MEDIUM	ACKNOWLEDGED	p. 86
M-02	Deposits, withdraws And RetentionRefunds Are Not EIP712 Compliant	Logical Error	MEDIUM	ACKNOWLEDGED	p. 87
M-02.2	Contract should be paused or removed right after unwind	Logical Error	MEDIUM	ACKNOWLEDGED	p. 88
M-03.2	Signatures cannot be created while operationOngoing is true	DoS	MEDIUM	ACKNOWLEDGED	p. 89
M-04.2	Minimum deposit amount are not checked on Admin orders	DoS	MEDIUM	ACKNOWLEDGED	p. 90
M-03	jUSDC Payback Through keeperPayBack Does Not Use Strategy Balance	Logical Error	MEDIUM	ACKNOWLEDGED	p. 91
M-04	jUSDC Cannot Enforce Payback	Logical Error	MEDIUM	ACKNOWLEDGED	p. 92
M-05	Hardcoded slippage	Logical Error	MEDIUM	ACKNOWLEDGED	p. 93
M-06	Pool is not always 50/50 leading to bad calculations.	Logical Error	MEDIUM	ACKNOWLEDGED	p. 94
M-07	newTotalUSD is based on borrowing non truncated amount	Logical Error	MEDIUM	ACKNOWLEDGED	p. 95
M-08	Execution feature disabled checks not implemented in admin flows	Validation	MEDIUM	ACKNOWLEDGED	p. 96

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-01	Keeper Can Be Made To Lose Gas On Failing Pending Deposits	Logical Error	LOW	ACKNOWLEDGED	p. 97
L-01.2	IncentiveReceiver receives less funds in KeeperPayback flow when USDC depegs	Validation	LOW	ACKNOWLEDGED	p. 98
L-02	Potential Native ETH Trapped	Logical Error	LOW	ACKNOWLEDGED	p. 99
L-02.2	Allowances not cleared while updating GMX variables and swappers in the RebalanceStrategy	Logical Error	LOW	ACKNOWLEDGED	p. 100
L-03.2	Sum of fees can exceed 100% and prevent execution	Validation	LOW	ACKNOWLEDGED	p. 101
L-03	Trapped ETH On keeperPayBack Call	Logical Error	LOW	ACKNOWLEDGED	p. 102
L-04	rebalanceAmounts not implemented in getGMWeight function	Logical Error	LOW	ACKNOWLEDGED	p. 103
L-05	Attacker can still DoS even with minimum amounts being applied	Logical Error	LOW	ACKNOWLEDGED	p. 104
L-07	Funds still trapped from overspending as well as refund from GMX	DoS	LOW	ACKNOWLEDGED	p. 105

CRITICAL **C-01**

PARTIALLY RESOLVED

Leverage Strategy DoS Through Depositing And Withdrawing Dust

CATEGORY	DoS
LOCATION	GMStrategy.sol:540
REVIEW	Main Review

— DESCRIPTION

When an user enters a position by depositing funds through the `LeverageRouter` contract and subsequently withdraws it, `GMStrategy.afterWithdrawalExecution` is called, swapping the long token for USDC.

At the beginning of the withdrawal process, the `action` variable in `LeverageStrategy` is set to 3. If the amount deposited is very small, and the long token is BTC, the withdraw fails before setting the variable back to its original state, without reverting the transaction.

This happens because 1 `SATS`, the smallest `WBTC` unit, is worth more than the smallest `USDC` denominator, and when `GMStrategy.afterWithdrawalExecution` is called by the GMX Withdrawal Handler, the Uniswap v3 call reverts due `amountSpecified` rounding down to zero.

The `action` state variable is supposed to be set back to 1 in an external call back to `LeverageStrategy` made past this point.

The transaction still goes through as the handler will catch the error and only emit an event, without reverting the call stack up until this point. This leaves the contract at the `action = 3` state and blocks most of its functions as they require the variable to be at an idle state of 1 to work.

The DOS can be resolved only by having governance call the `enforceAction` function, to reset the contract back but an attacker can continuously re-exploit it.

— RECOMMENDATION

Implement a minimum withdrawal amount that must be met when creating a withdrawal. This can be dynamic in case the protocol need to change it.

C-01 Leverage Strategy DoS Through Depositing And Withdrawing Dust

CONTINUED

— RESOLUTION

Status: Partially resolved.

<https://github.com/Jones-DA0/jones-gm/pull/21>

CRITICAL C-02

RESOLVED

Leverage Strategy Debt Pay Back Fails Due To Insufficient Allowance

CATEGORY Logical Error
LOCATION LeverageStrategy.sol:907
REVIEW Main Review

DESCRIPTION

When the leverage strategy needs to pay back a part of its debt to the stable vault, the `payBack` function is invoked by an operator. The `LeverageStrategy.payBack` function calls the `IUnderlyingVault.payBack` function and the underlying stable vault transfers the tokens.

For payback to be successful, the underlying stable vault (`stableVault`) requires allowance. The vault is approved with infinite allowance during initialization [here](https://github.com/GuardianAudits/jgm-pocs-1/blob/8e661971ca0a893d966192af907342d667e8e38d/src/leverage/LeverageStrategy.sol#L125), resulting in normal operation flow.

However, `LeverageStrategy` contract also has an internal `_repayStable` function, which is called during `unwind`, `leverageDown` and some `withdraw` actions.

```
function _repayStable(uint256 _amount) internal returns (uint256) {  
    uint256 amountToRepay = _amount > stableDebt ? stableDebt : _amount;  
    stable.approve(address(stableVault), amountToRepay);  
    stableVault.payBack(amountToRepay, 0);  
    // ...  
}
```

This function approves the `stableVault` again with the value of `amountToRepay` (even though it already has infinite allowance), and immediately consumes the same amount of allowance. After this action, the infinite allowance will basically be overwritten and zeroed out.

`LeverageStrategy.payBack` function will always revert due to insufficient allowance, after `_repayStable` is called even once. This results in the complete blocking of payback functionality, a core feature.

Although the contract has a way to re-approve the underlying vault by calling the `forceApproval` function, it is callable only by governance and calling it before every payback is not feasible.

C-02 Leverage Strategy Debt Pay Back Fails Due To Insufficient Allowance

CONTINUED

RECOMMENDATION

Do not re-approve the `stableVault` in the `_repayStable` function since the vault is already approved.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/23>

CRITICAL C-03

RESOLVED

Attacker Can DOS GM Strategy By Abusing Blacklisted USDC Addresses

CATEGORY	DoS
LOCATION	LeverageStrategy.sol:386
REVIEW	Main Review

DESCRIPTION

Withdrawing funds is a 2-Step process:

1. The user calls `createWithdrawal`
2. The GMX keeper bot executes the order

No other action in the system is allowed till this process is completed.

The last state change of the second step is the transfer of USDC to the user. If the user is blacklisted by USDC the call from GMX will revert and retrying will also fail.

This enables the following attack path:

- Attacker calls `createWithdrawal` with a receiver that is on the USDC blacklist
- `depositCallback` will revert when attempting to transfer USDC to the blacklisted address
- `operationOnGoing` is stuck as true preventing future deposits and withdrawals

RECOMMENDATION

Check if the receiver of the withdrawal is blacklisted with `USDC.isBlacklisted(receiver)`.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/17>

CRITICAL **C-04**

RESOLVED

Function `isInRange` will not catch out of range values

CATEGORY	Logical Error
LOCATION	GMStrategy.sol:962
REVIEW	Main Review

DESCRIPTION

The `isInRange` function should be used in the withdrawal process to compare the expected return to the actual return (slippage check). But as we can see it doesn't and instead compares the given parameters to themselves:

```
if (_idealUSDC < applySlippage(_idealUSDC) || _actualUSDC > addSlippage(_actualUSDC)) { revert OutOfRange(); }
```

It checks if the ideal USDC value is bigger than itself - x% and if the actual USDC value is smaller than itself + x%. This will always be true and therefore nothing is checked at all.

RECOMMENDATION

Compare the parameters against each other, not to themselves.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/18>

CRITICAL C-05

RESOLVED

Value Extracted During Faulty Rebalance

CATEGORY	Logical Error
LOCATION	GMStrategy.sol:716-718
REVIEW	Main Review

DESCRIPTION

When a keeper starts a rebalance by calling `RebalanceStrategy.startRebalance`, the function will set the `operationOnGoing` flag in the `GMStrategy` contract in order to prevent new deposits and withdrawals from being submitted.

This is done because the rebalance is an operation sensitive to the quantity of funds in the strategy contract. At the start of a rebalance, the `GMStrategy.rebalanceInput` function is called, which sets the flag to true.

However, the flag is only set when the rebalance stage is 1 (`rebalanceStage == 1`), and the `nextRebalanceStage` function, which increments the stage, is called before the `rebalanceInput` function is called, incrementing the `rebalanceStage` variable to 2, thus preventing the flag from being activated.

As a consequence, deposits and withdrawals will be enabled as normal during the rebalance.

An attacker can look for rebalances and when one starts, after the funds were moved from the `GMStrategy` then they deposit into the strategy. By doing this, after the rebalance is then finished, they withdraw with a profit since finishing the rebalance implies sending back the new GM tokens to the strategy, effectively increasing the value of the attacker's deposit.

RECOMMENDATION

In `GMStrategy.rebalanceInput` function, set `operationOnGoing` to true regardless of current rebalance stage, since this function is only called at the start of a rebalance.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/29>

CRITICAL C-06

RESOLVED

Borrowed USDC Is Counted As User Deposit

CATEGORY Logical Error
LOCATION LeverageStrategy.sol:263-267
REVIEW Main Review

DESCRIPTION

When a deposit is made, `jGM` shares will be minted to the user based on `data.shares`. The issue arises when a deposit is leveraged because `data.shares` includes the additional borrowed amount. However, when the deposit is not leveraged, `data.shares` only reflects the amount the user provided initially.

This discrepancy creates a situation where users depositing with leverage receive more shares per asset than users making non-leveraged deposits, despite facing the same risk associated with `jGM`.

This discrepancy directly leads to profit at the expense of the other users for those that deposit while leverage is below target, as the excess shares that were minted can then be withdrawn.

RECOMMENDATION

To address this issue, modify the minting of `jGM` shares to be based on the `amount` stored in the `callbackData` mapping. This change will exclude the leveraged amount from the calculation:

```
shares = _jGM.mulDivDown(callback.amount, callback.amount + callback.leverage);
```

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/19>

CRITICAL **C-07**

PARTIALLY RESOLVED

Full Protocol DOS Via Spamming Operations

CATEGORY DoS
LOCATION GMStrategy.sol
REVIEW Main Review

— DESCRIPTION

The entire `jGM` system does not allow more than 1 operation (withdraw/deposit) to function at a time. During the time from when an operation was initiated on GMXv2 and when it was done (the [keepers execute the request](https://gmx-docs.io/docs/trading/v2/#execution-fee)) no other operation is allowed.

This odd design, combined with the lack of a minimum deposit/withdraw amount, allows an attacker to continuously spam deposits to block any operations to be done on the system. The attack is cheap and would result in a complete system block.

— RECOMMENDATION

Implemented a minimum withdraw/deposit amount. Also consider completely redesigning the system to not have a global lock on each operation.

— RESOLUTION

Status: Partially resolved.

<https://github.com/Jones-DAO/jones-gm/pull/39>

CRITICAL C-08

RESOLVED

Attacker Can Repurpose Signatures To Damage Protocol

CATEGORY	Logical Error
LOCATION	GMRouter.sol:174-183 GMStrategy.sol:492
REVIEW	Main Review

DESCRIPTION

The vulnerability is a combination of 3 issues with the core issue being that whenever a user interacts with the protocol, the off-chain systems must validate a part of the input data and sign it, in order to protect against manipulations. The information within a signature that is validated is represented by the `IGMRouter.Data` structure. This structure, however, does not include the type of operation that was created or the input token amount.

One attack scenario

When a user withdraws, the amount of `Leveraged jGM` shares he burns is returned to him in equivalent USDC. The information in the signed `IGMRouter.Data` is created as such that he receives a fair amount but there are no checks on-chain that the equivalent burned `Leveraged jGM` is roughly the same as that indicated by the `IGMRouter.Data` structure, as validations are off-chain only.

By combining the above, an attacker can:

- say the TVL in the strategy is 1,000,000 USD distributed in GM tokens according to the targeted weights
- initiate a deposit in the dapp of 1,000,000 USDC equivalent
- the dapp will generate a signature and a `IGMRouter.Data` structure with a buy equivalent of GM tokens, to equal 1,000,000 USD in value, distributed in GM tokens according to targeted weights
- the dapp will now provide the data to the "user" for him to call the protocol with
- the attacker then takes the entire `IGMRouter.Data` and signature and passes it directly, bypassing the dapp, to a withdraw instead, setting an insignificant amount of `Leveraged jGM` tokens
- the withdraw will be initiated successfully, due to no value checks on the passed data in the `strategyWithdraw` function but will fail after GMX has fully closed all the GM strategy positions due to the faulty signature data in the `afterWithdrawalExecution` callback

C-08 Attacker Can Repurpose Signatures To Damage Protocol

CONTINUED

Although it blocks an attacker from stealing the funds it does not stop him from completely withdrawing from GMX the entire `GM` Strategy balance.

This also results in loss of yield from the unstaked GM tokens as well as blocking all further withdraws, since there are no more GM tokens to withdraw, requiring manual intervention to resolve.

Another attack path is regarding deposits. In deposit again there is no validation that the passed USDC is equivalent to the value to be bought in GM tokens.

Attack scenario:

- attacker waits for stables to accumulate in the contract
- initiates a deposit of the exact amount of stables existing in the contract
- the dapp will generate a signature and a `IGMRouter.Data` structure with a buy equivalent of GM tokens
- the attacker then again takes the entire `IGMRouter.Data` and signature and passes it directly, bypassing the dapp, while depositing dust USDC.
- due to deposit implementation attacker only gets an equivalent share amount to the dust he deposited but the entire USDC in the strategy was added to GMX

For the second scenario, an attacker can force a "compound" action instead of a "payBack". Example, the attack might be done exactly after a `keeperPayBack` call, where the intention to initiate a debt payback is clear (and made in a 2 step manner).

— RECOMMENDATION

Modify the `IGMRouter.Data` structure to include amount and operation type (use an Enum for operations).

In the `strategyWithdraw` function from the `GMStrategy` contract, validated that the approximate USDC gains from withdrawing the all GM tokens from the `GMDData` array are equal to the `_assets` amounts. Meaning also create the equivalent `_isInRange` check before initiating the withdraw call.

In the `strategyDeposit` function from the `GMStrategy` contract, validate that the passed `USDC` `_amount` is equal to that used to buy all the GM tokens.

— RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/18>

CRITICAL C-09

RESOLVED

Attacker can cause DoS by setting receiver to address(0)

CATEGORY DoS
LOCATION LeverageRouter.sol:67
REVIEW Main Review

DESCRIPTION

When a user calls the `createDeposit` function, they have the option to choose any address as the receiver. After the deposit completes, the `afterDepositExecution` function triggers a `mint` operation to create shares for the designated receiver. However, attempting to mint shares to `address(0)` causes a revert. This revert leads to the contract state variable `operationOnGoing` remaining true, which is required to be false for both deposits and withdrawals to proceed. This situation will prevent future deposits or withdrawals.

Similarly, the `withdrawalCallback` function also encounters a revert when attempting to transfer assets to `callback.receiver`, resulting in the same outcome.

RECOMMENDATION

Check and require that `receiver` does not equal `address(0)` in both `createDeposit` and `createWithdrawal` functions.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/16>

CRITICAL C-10

PARTIALLY RESOLVED

Liquidated Users Can Steal Future Deposits

CATEGORY	Logical Error
LOCATION	LeverageStrategy.sol: 264
REVIEW	Main Review

— DESCRIPTION

When the governor or keeper calls `LeverageStrategy.unwind`, the contract can become unusable due to the action state being stuck at 2 after a standard user deposit, as discussed in a separate issue.

However, even in the scenario where the responsible actors unstuck the contract state by enforcing action back to 1 and providing it some funding, so operations return to normal, previous users liquidated by the unwind operation will still own their leverage vault shares.

Due to this reason, even when a new user deposits in the system after the governance/keeper intervention, a previously liquidated user can steal some of the funds provided by the new user, by submitting a withdrawal with their leveraged vault shares.

The amount of funds the previously liquidated user can "claw back" will depend on the total underlying value the keeper has injected into the protocol; in some cases, it can be greater than their previous deposit, and in other cases, it will be less.

On top of this, when the new user attempts to withdraw his funds, the callback from GMX will revert without reverting the whole transaction, causing the action to be in an invalid state again.

The provided PoC showcases a scenario where the governance will provide fresh new underlying funds immediately after an unwind, a new user enters, a previous user steals a portion of funds, and the new user attempts to exit all their shares unsuccessfully.

— RECOMMENDATION

In order to not be able to manipulate the system, liquidated users should not own leveraged vault shares, so the unwind operation must burn them. Otherwise, migrate the leverage vault.

C-10 Liquidated Users Can Steal Future Deposits

CONTINUED

— **RESOLUTION**

Status: Partially resolved.

CRITICAL C-11

PARTIALLY RESOLVED

Attacker can call cancellation callback to deplete strategy of GM tokens

CATEGORY Validation

LOCATION <https://github.com/GuardianAudits/jgm-pocs-2/blob/master/src/gm/strategies/GMStrategy.sol#L378>

REVIEW Main Review

DESCRIPTION

The `afterDepositCancellation` and `afterWithdrawalCancellation` functions trigger the bot to retry a cancelled order by calling either `executeSingleDeposit` or `executeSingleWithdraw` based on the emitted data from the cancellation. However, any user can set the `callbackContract` to the `GMStrategy` contract on their own malicious contract.

This allows an attacker to orchestrate a guaranteed-to-fail withdraw with the `GMStrategy` as the callback contract. Subsequently, when the cancellation callback is invoked, the keeper will retry the order with the GM tokens from the `GMStrategy`.

This vulnerability enables an attacker to deplete the strategy of its GM tokens, leaving the underlying temporarily stuck in the `GMStrategy`. As a result, users who withdraw during this time will experience a substantial loss, as their JGM shares will be backed by little to no actual GM tokens.

RECOMMENDATION

Utilize the `key` parameter to verify that the cancellation is originating from an order initiated by `jGM`. This can be achieved by checking if `keys[key]` is empty and reverting if it is.

Additionally all GMX callbacks should explicitly check that the `key` is from an order initiated by `jGM`.

RESOLUTION

Status: Partially resolved.

<https://github.com/Jones-DA0/jones-gm/pull/20>

CRITICAL C-12

PARTIALLY RESOLVED

Attacker Can cause DoS by Intentionally exceeding range check

CATEGORY	DoS
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/master/src/gm/strategies/GMStrategy.sol#L962
REVIEW	Main Review

DESCRIPTION

When a user creates a withdrawal, they will transfer GM tokens to the GMX withdrawal handler. GMX will then record the transfer by taking the `balanceOf` of the contract. This quirk allows for users to alter the amount used when creating an order from what was initially intended when the `createWithdrawal` function was called. In some cases, this is not a bad thing as it results in more underlying tokens received.

However, with jGM's current design, this action results in the `operationOnGoing` variable being stuck as true, preventing future deposits and withdrawals. The way an attacker would exploit this would be to execute a transaction that does two things:

- First, transfer a small amount of GM tokens to the `withdrawHandler`.
- Then call the `createWithdrawal` function in jGM.

What this will do is unexpectedly increase the amount of jGM tokens used to withdraw and consequently the amount of underlying tokens received. Leading to a revert in the `afterWithdrawalExecution` function when the `_isInRange` function is called and compares the `data.assets` value to `data.usdc`.

The reason this will revert is that `data.usdc` is based on the greater than expected amount of GM tokens, while `data.assets` is based on what was sent by the protocol, which was not aware of the attacker's initial transfer.

RECOMMENDATION

Alter the check so that if `data.usdc` is above the acceptable range, it is truncated to the maximum acceptable value. The excess can then be transferred to a protocol-approved address.

RESOLUTION

Status: Partially resolved.

C-12 Attacker Can cause DoS by Intentionally exceeding range check

CONTINUED

<https://github.com/Jones-DA0/jones-gm/pull/28>

CRITICAL C-13

RESOLVED

Compound and Rebalance do not increase the nonce

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/8e661971ca0a893d966192af907342d667e8e38d/src/gm/strategies/GMStrategy.sol#L643-L703
REVIEW	Main Review

DESCRIPTION

The `compound` as well as the `rebalance` functions do not increase the nonce value. This enables the following attack paths:

Compound:

- Attacker fetches a signed data param from the keeper bot every few seconds
- Keeper bot calls the `compound` function
- After the compound flow ends the attacker calls `createDeposit` with the stale data param from before the keeper bot called `compound`
- As the nonce was not increased the stale data param is still valid and the attacker receives excess shares as the stale `usdTotalValue` messes with the share calculation

Rebalance:

- Keeper bot calls the `rebalance` function
- The attacker already watched out for this event and fetches the signed data param between the withdraw and the deposit step of the rebalance flow
- After the rebalance flow ends the attacker calls `createDeposit` with the stale data param from the middle of the rebalance flow
- As the nonce was not increased the stale data param is still valid and the attacker receives excess shares as the stale `usdTotalValue` messes with the share calculation

RECOMMENDATION

Increase the nonce in the `GMRouter` when calling `compound`, or `rebalance`.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/51>

CRITICAL C-14

PARTIALLY RESOLVED

Attacker can deposit while ongoingOperation is true to mint extra shares

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/master/src/gm/GMRouter.sol#L92
REVIEW	Main Review

DESCRIPTION

When a deposit is created the keeper will retrieve the `usdTotalValue`. This value is the USD value of all the GM tokens in the GMStrategy. The `usdTotalValue` is used to calculate the right amount of shares to be minted to the user as well as the amount of IJGM (index) tokens. Because `usdTotalValue` is the denominator when calculating the shares to be minted, meaning the smaller `usdTotalValue` is the more shares that will be minted.

The issue here is that after a deposit is created, and before it is executed, the keeper will retrieve an inaccurate value for `usdTotalValue`. This is because the total amount of GM tokens in the vault do not include the incoming GM tokens from the most recent deposit, as the deposit has not been executed yet. Because of this when a user pings the keeper to create a deposit `_data` object while `operationOngoing` is true, they can then use that outdated `usdTotalValue` value on their deposit. Because the outdated `usdTotalValue` is smaller than it should be, it will mint more shares to themselves at the expense of the other users.

RECOMMENDATION

Create a pending USD value when deposit are created, and add this to the `usdTotalValue`. After a deposit is executed set the pending to zero. Some version of the following would mitigate this attack.

In the `strategyDeposit` function:

```
pendingIncomingUSD = pendingIncomingUSD + ((minAmount * 1e12 / 0.995e12) *  
GMPrice(info.gm.oracle, info.gm.stalePeriod));
```

In the `afterDepositExecution` function:

```
pendingIncomingUSD = 0;
```

In the `totalValue` function:

```
return (_totalValue + pendingIncomingUSD) / 1e18;
```

C-14 Attacker can deposit while ongoingOperation is true to mint extra shares **CONTINUED**

— RESOLUTION

Status: Partially resolved.

<https://github.com/Jones-DA0/jones-gm/pull/38>

CRITICAL C-15

PARTIALLY RESOLVED

Attacker can cause DoS by overriding gmxData key

CATEGORY	DoS
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/master/src/gm/strategies/GMStrategy.sol#L466
REVIEW	Main Review

DESCRIPTION

In both the `strategyDeposit` and `strategyWithdraw` functions the data relevant to the deposit and withdraw is stored in the `gmxData` mapping. The key for this mapping is a combination of the `receiver` as well as `block.timestamp`. The `receiver` will always be determined by the user who is creating the deposit. And the `block.timestamp` will be the current time in seconds.

One way Arbitrum differs from some other chains is that Arbitrum can produce multiple blocks per second, in many cases four blocks in a single second. Because the key to the `gmxData` mapping is made up of the current time, and because Arbitrum can have multiple blocks in a second it is possible to override one order with another.

Even with the asynchronous nature of the GMX keepers it is possible to have two withdraws close enough together that the second withdraw will override the first. The main impact of this is that the `data.usdc` will start as a non-zero value on the second withdraw, which results in `data.usdc` being larger than the amount of underlying in the contract.

Because of this the attempt to transfer `data.usdc` will fail due to insufficient funds. When the callback fails the protocol will not be able to perform future deposits or withdraws as the `operationOngoing` will still be true. With high enough traffic on the protocol, or a malicious sequence of withdraws, the protocols functionality can be halted.

RECOMMENDATION

Either add a nonce when setting `info.opHash` in both the `strategyDeposit` and `strategyWithdraw` functions. This will keep the value unique even when actions are made within the same `block.timestamp`.

Or reference the Arbitrum block number instead of `block.timestamp`. This will ensure a unique hash on every order.

C-15 Attacker can cause DoS by overriding gmxData key

CONTINUED

— RESOLUTION

Status: Partially resolved.

<https://github.com/Jones-DA0/jones-gm/pull/40>

HIGH **H-01**

PARTIALLY RESOLVED

GM Strategy Token Weight Is Errantly Determined

CATEGORY	Logical Error
LOCATION	GMStrategy.sol:189
REVIEW	Main Review

DESCRIPTION

The GM index composition of the `jGM` vault is monitored to ensure it remains within a predetermined range. This is crucial to maintaining a healthy GM system and protocol robustness.

The target weight is read directly from the `GMStrategy` contract using the `tokenWeight` function. This function incorrectly calculates the weights and returns them as 30 decimals values, instead of less than 12 decimals, [as they are intended](https://github.com/GuardianAudits/jgm-pocs-1/blob/master/src/gm/strategies/GMStrategy.sol#L159-L161).

The error appears when multiplying the in-question Strategy GM token balance value, by omitting to divide by `1e18`

```
uint256 currentValue = IERC20(gm.token).balanceOf(address(this)) * GMPrice(gm.oracle,  
gm.stalePeriod);
```

Since this value is instrumental in maintaining the balance of the system and triggering a rebalance, it is crucial to have it correctly determined.

RECOMMENDATION

Divide the `currentValue` with `1e18` before dividing it to the total value.

RESOLUTION

Status: Partially resolved.

<https://github.com/Jones-DA0/jones-gm/pull/22>

HIGH **H-02**

ACKNOWLEDGED

Withdraws Over Leverages The System

CATEGORY	Logical Error
LOCATION	LeverageStrategy.sol:315-321
REVIEW	Main Review

DESCRIPTION

When a user initiates a withdraw from the system via the `LeverageRouter.createWithdrawal` function, execution reaches the `LeverageStrategy` contract where the current leverage is checked in order to determine if there is a need to deleverage.

This check is incorrectly implemented because instead of comparing if the current leverage is higher then target leverage, it checks if it is higher then maximum leverage. This means that even if the system is above the target leverage, until it reaches the maximum no automatic actions will be taken.

```
uint256 maxLev = getMaxLeverage();  
// ...  
if (currentLev > maxLev) {
```

With the current implementation there is always a tendency to over-leverage the system and always be above target leverage. This reduces the overall system health introduction economical risk to users.

RECOMMENDATION

Change withdraws that if they are above target leverage to deleverage.

RESOLUTION

Status: Acknowledged.

HIGH **H-03**

PARTIALLY RESOLVED

Incorrect Underlying jGM Calculations Due To Fees And Slippage

CATEGORY	Logical Error
LOCATION	LeverageStrategy.sol:900-902
REVIEW	Main Review

DESCRIPTION

Throughout the protocol, in the `LeverageStrategy` contract there is a `_getjGMToPayback` function which is used to estimate how much `jGM` tokens should be withdrawn in order to get the passed amount of USDC (debt).

```
function _getjGMToPayback(uint256 _debt, uint256 _totalUSDValue) private view returns (uint256) {  
    return GMStrategy.addSlippage(GMViewer.getPreviewDeposit(_debt, _totalUSDValue));  
}
```

This function retrieves the required amount and increases it with the set GM Strategy slippage (defaulted to 0.5%). This is done to ensure that deposit with swaps that reach the slippage limit would still work since the withdraw amount has taken this into consideration.

There are 3 issues with the current implementation:

- the amount increase due to possible slippage is added on the entire amount, where, on GMX withdraws, the liquidity is returned in 50% short token (USDC) and 50% long tokens. As such, only the 50% liquidity in long token is swapped and should apply the slippage increase.
- the amount increase to equivalent fee taking is incorrectly added. Since out of the given amount a percentage % would be withdraw, we need have the amount increase as $\text{amount} = \text{amount} * \text{BASIS} / (\text{BASIS} - \text{slippage})$ so that when the fee is applied, the exact `amount` value is returned. `addSlippage` incorrectly adds the %, resulting in less tokens.

Example scenario:

Say the swaps reach the slippage limit of `0.5%`, meaning that `99.5%` of `amount` was returned. For an `1000` initial amount, that was increased with `0.5%` means `1005` and `99.5%` of `1005` is `999.975`, less than `1000`. The correct way is to have the amount increased as: $1000 * 100 / (100 - 0.5) = 1005.025125$ and out of that amount, if we get only `99.5%` we then get exactly `1000`

H-03 Incorrect Underlying jGM Calculations Due To Fees And Slippage

CONTINUED

• The third issue is that, in order to correctly simulate a GMX withdraw (position close), a fee must also be added to also simulate this. The current GMX position open or close fee is `0.07%` in the `jGM` case, [because it only deposits or withdraws one pair](https://gmx-docs.io/docs/trading/v2/#open--close-fees). This fee must also be taken into consideration for exact calculations by applying it in the same manner.

Because of the way the current `_getjGMToPayback`, and implicit leverage is calculated, overall system appears more leveraged then it is in reality.

— RECOMMENDATION

After calculating the pure amount required to withdraw, increase it, using the above mentioned formula, to compensate the GMX fee. On half of the resulting amount increase, again using the above formula, the amount to compensate for the swap slippage requirement. Finally return the sum of first half and the increased half as the underlying.

— RESOLUTION

Status: Partially resolved.

<https://github.com/Jones-DAO/jones-gm/pull/42>

HIGH **H-04**

RESOLVED

Depositing After Unwind Reverts

CATEGORY Validation
LOCATION LeverageStrategy.sol:264
REVIEW Main Review

DESCRIPTION

When the governance or keeper deleverages the system by calling `LeverageStrategy.unwind`, a withdraw order is created, passing all the `jGM` balance as shares.

By the end of this operation, the `LeverageStrategy` contract is left with no `jGM` shares. However, the `LeverageVault` shares owned by the users are left untouched.

Due to this reason, when an user follows through with a deposit, the operation will work as intended and eventually reach `depositCallback`, where it will revert, as the `jGM` shares to be minted are multiplied by the total supply of `LeverageVault` shares, which won't be zero, and divided by `callback.underlyingjGM`, which will be zero after an unwind.

The contract is left at action state 2, and although the operator of `jGMIndex.sol` and the governance can solve this issue by directly minting some shares to the leverage strategy contract, and governance can enforce the action back to 1, deposits and withdrawals will stop working until then.

RECOMMENDATION

Change the check at the provided line to the following:

```
if (callback.supply > 0 && callback.underlyingjGM > 0) {
```

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/49>

HIGH **H-05**

PARTIALLY RESOLVED

Execution features may be disabled

CATEGORY	Validation
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/master/src/gm/strategies/GMStrategy.sol#L271
REVIEW	Main Review

— DESCRIPTION

The GMX Keeper has the authority to disable specific features when necessary, such as executing deposits and withdrawals. If these features are disabled, attempts by keepers to execute orders will result in reverts, causing the orders to be canceled. Subsequently, if the JGM keeper tries to recreate the deposit with altered parameters for a successful order, it will also fail due to the disabled feature.

This situation presents a critical issue as JGM users will be unable to interact with the protocol entirely, even if only one of deposits or withdrawals is disabled. This scenario is particularly concerning because if deposits are disabled, users should ideally withdraw their funds promptly. However, whether intentionally or unintentionally, a user can effectively halt all protocol operations by initiating a deposit or withdrawal while the execution functionality is disabled.

— RECOMMENDATION

Implement checks in the deposit and withdrawal functions to verify that deposit executions are enabled for deposits and that withdrawal execution is enabled for withdrawals. These checks will ensure that users can only perform these actions when the respective execution features are active, preventing disruptions to protocol operations due to disabled functionalities.

— RESOLUTION

Status: Partially resolved.

<https://github.com/Jones-DAO/jones-gm/pull/52>

HIGH **H-06**

PARTIALLY RESOLVED

User can always use stale price

CATEGORY Validation

LOCATION <https://github.com/GuardianAudits/jgm-pocs-2/blob/master/src/gm/strategies/GMStrategy.sol#L271>

REVIEW Main Review

— DESCRIPTION

To deposit and withdraw funds as well as for many other functions the system receives a signed data param that includes price data for calculations. This data param is valid for a given amount of time after it was signed.

An attacker can abuse this by constantly fetching the signed data param until an event occurs that leads to dangerous stale data in less than 60 seconds.

Attack flow:

- Attacker constantly fetches a signed data param every few seconds
- A big price change occurs for example a trader takes a large profit on GMX (this could be the attacker or anyone else)
- The attacker uses the stale signed data param to deposit or withdraw funds and profits from the price change

This attack can always be done to extract value from the other users, however only profitable for the attacker when the gain from using stale data outweighs the fee charge for doing so.

— RECOMMENDATION

Consider fetching price data on chain to ensure accurate pricing.

— RESOLUTION

Status: Partially resolved.

<https://github.com/Jones-DAO/jones-gm/pull/57>

MEDIUM **M-01**

RESOLVED

Manipulated swap pool will cause DoS on withdrawals

CATEGORY DoS

LOCATION <https://github.com/GuardianAudits/jgm-pocs-2/blob/master/src/gm/strategies/GMStrategy.sol#L271>

REVIEW Main Review

DESCRIPTION

When there is any price manipulation in a pool that is being used to swap the long token for the short token the swap attempt will revert. This is good in most cases, however when the swap fails in the `afterWithdrawalExecution` function it will revert and result in the `operationOnGoing` variable being stuck as true.

An attacker can exploit this by performing a large swap between the creation and execution of the withdrawal. Causing the before mentioned revert and preventing any further operations. In addition to an attacker specifically targeting JGM, any large swap(s) will cause this situation. This includes non malicious swaps. As soon as there is a price deviation of 0.1% or greater the price manipulation check will fail.

RECOMMENDATION

Because the `_isInRange` function will ensure enough underlying was received, use a swapper contract that does not check for manipulation, as the `_isInRange` function will ensure an acceptable amount of assets are returned.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/56>

MEDIUM **M-02**

PARTIALLY RESOLVED

GMRouter Deposits And Withdraws Are Not EIP712 Compliant

CATEGORY Logical Error
LOCATION GMRouter.sol:177-178
REVIEW Main Review

DESCRIPTION

When initiating a deposit or withdraw through the `GMRouter` contract, a signature must also be passed that will be checked in the `_verifyParams` to be from the correct signer which will also validate the provided input data.

The router inherits the `EIP712Upgradeable` contract but it does not properly handler the signature verification in relation to the EIP-712 standard. The `[typeHash](https://eips.ethereum.org/EIPS/eip-712#rationale-for-typehash)` of the `IGMRouter.Data` structure is never calculated when it is verified in `_verifyParams`:

```
address _signer =  
ECDSA.recover(_hashTypedDataV4(keccak256(abi.encode(nonce, keccak256(abi.encode(params))))),  
signature);
```

As such, while the operations work, the transaction lacks the clarity which comes to the EIP. Depending on the operator, it may be deter usage.

RECOMMENDATION

Correctly implement the `[EIP-712](https://eips.ethereum.org/EIPS/eip-712)` verification by adding the `typeHash` while also adding the nonce in the signature. The structure itself can have the nonce and it is to be checked to be the "next up" after the validation succeeds. As the EIP indicates, the `typeHash` of the `IGMStrategy.GMData` itself must also be pre-calculated and included in the `IGMRouter.Data`.

RESOLUTION

Status: Partially resolved.

<https://github.com/Jones-DA0/jones-gm/pull/50>

MEDIUM **M-03**

PARTIALLY RESOLVED

Potential Tokens Trapped On GM Strategy Deposits

CATEGORY Validation
LOCATION GMStrategy.sol:296-297
REVIEW Main Review

DESCRIPTION

When an operator deposits into the GM strategy through the router, a specific amount of USDC and native ETH is transferred to the strategy and then the strategy is notified of the deposit amounts

```
///  
@notice Transfer Assets to Strategy  
USDC.transferFrom(msg.sender, address(strategy), _assets);  
///  
@notice Trigger Deposit in Strategy  
strategy.strategyDeposit{value: msg.value}(_receiver, _assets, shares, data.gmData);
```

The strategy then incorrectly never uses the `_amount` (amount of USDC to be deposited) argument or the `msg.value` received to determine if there were sent more or less tokens.

```
gmRouter.sendWnt{value: _data[i].executionFee}(gmxDepositVault, _data[i].executionFee);  
gmRouter.sendTokens(USDC, gmxDepositVault, _data[i].amount);
```

If less than indicated native ETH (`msg.value`) was sent or less `USDC` the operation would revert, but if more is sent then the funds remain blocked in the strategy contract until `emergencyWithdraw` is called by the contract owner. During this time, any potential yield from the leftovers is also lost.

RECOMMENDATION

Before iterating through the `GMDData` entries, in the `strategyDeposit` function, cache the `msg.value` and `_amount` amounts and on each iteration decrease both cached values. If at the end of the entire function call these amounts are both not 0 then too much tokens were sent and the operation should either revert or send the extra tokens to the `receiver`.

RESOLUTION

Status: Partially resolved.

M-03 Potential Tokens Trapped On GM Strategy Deposits

CONTINUED

<https://github.com/Jones-DA0/jones-gm/pull/18>

MEDIUM **M-04**

RESOLVED

Executing Partial Or Premature Pending Operations Blocks Further Rebalances

CATEGORY	Logical Error
LOCATION	RebalanceStrategy.sol:258-310 RebalanceStrategy.sol:452-507
REVIEW	Main Review

DESCRIPTION

When a rebalance is started, a number of GMX withdraws are first initiated. If any fail, then they are added to a pending withdraws array. Then they can be retried via the `executePendingWithdraw` function. The same logic applies when, at the last stage of a rebalance, the new deposits are done but by calling `executePendingDeposit`.

Both `executePendingWithdraw` and `executePendingDeposit` do not clear the pending operations after created, as such if any one of the following situation happens then the pending operation cannot be further retried and funds remain blocked in the case of deposits:

1. the `executePending*` functions are called before all operations that fail are stored

Example scenario: 4 deposits are attempted, 2 succeed, 1 fails, and before the last one will also fail `executePendingDeposit` is called. Because of the check that `pendings[i].amount != _deposits[i].amount`, and the issue that the `pendings` array is never cleared, another call will will revert since the `pending[0].amount` (which was already sent to GMX on the first call) is again attempted again to be sent to GMX, resulting in either a revert there or on the next pending deposit.

1. the `executePending*` functions are called with a corresponding withdraw/deposit array less then the total pending operations.

Exactly as the above case, and due to a missing check that either `GMDeposit` or `GMWithdraw` arrays are the same length as the pending operations, calling the function once works but calling it a second time will revert it.

In the case of withdraws, `withdrawCounter` does not reach zero, and calling `afterWithdrawalExecution` via keeper to unblock rebalancing cannot be done since the `key` entry from the `keyToRebalance` mapping has already been deleted in `afterWithdrawalCancellation`. Leaving the GM tokens blocked in the contract and no way to go to the next rebalance.

In the case of deposits, similar as with withdraws, `pendingDeposits` does not reach zero on its own and calling `afterDepositExecution` by the keeper does not help as the

M-04 Executing Partial Or Premature Pending Operations Blocks Further Rebalances

CONTINUED

`keyToRebalance` entry has already been deleted in `afterDepositCancellation`. In this case USDC also remains blocked in the contract as well as the rebalance blocked.

In both cases funds are lost, rebalances are frozen leaving the team with the only option to initialize a contract upgrade.

— RECOMMENDATION

In both `executePendingWithdraw` and `executePendingDeposit` functions, remove the element from the pending arrays. Add an emergency withdraw function in case similar, unforeseen situations appear.

— RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/24>

MEDIUM **M-05**

PARTIALLY RESOLVED

Trapped ETH On keeperPayBack Call

CATEGORY Validation
LOCATION LeverageStrategy.sol:696-737
REVIEW Main Review

DESCRIPTION

When a keeper calls the `keeperPayBack` function from the `LeverageStrategy` contract, if the contract does not have enough stables, a GMX withdraw routine will be initiated. Within that GMX withdraw routine, several GM tokens may be withdrawn. For each GM token, the keeper must provide enough native ETH so that GMX's own keeper finalizes the withdraw.

With regards to the way native ETH is handled in this function, there are 2 issues:

- there is not check that more ETH was not mistakenly sent, then the required `gmxCoincentive` value
- if the function is called but, by coincidence there are enough stable tokens then the function secedes any ETH sent to it by the Keeper remains blocked, until it will be extracted via the `emergencyWithdraw` function.

RECOMMENDATION

Before calling the `GMRouter.createWithdrawal` function, validate that the `msg.value` is equal to the calculated `gmxCoincentive` amount.

If the functions determines that there are enough stables in it (`expectedStables == 0`) then either revert or send the native ETH back to the caller.

RESOLUTION

Status: Partially resolved.

<https://github.com/Jones-DA0/jones-gm/pull/33>

MEDIUM **M-06**

RESOLVED

Leverage Down On Zero Underlying Leaves Action Uncleared

CATEGORY Logical Error
LOCATION LeverageStrategy.sol:852 LeverageStrategy.sol:874
REVIEW Main Review

DESCRIPTION

When a keeper calls the `LeverageDown` function, if the system has no underlying tokens, the execution of the function ends while leaving the action set to 5.

Having the action uncleared blocks all other leverage strategy operations and requires governance intervention to fix via calling the `enforceAction` function to reset the flag.

RECOMMENDATION

Set the action state to 5 (leverage down) only just before calling the `GMRouter.createWithdrawal` function.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/34>

MEDIUM **M-07**

RESOLVED

New Leverage Incorrectly Calculated During Leverage Up

CATEGORY Logical Error
LOCATION LeverageStrategy.sol:801-808
REVIEW Main Review

DESCRIPTION

The keeper has the ability to increase the leverage by borrowing more tokens and depositing them into GMX via the `leverageUp` function. The new leverage can not be smaller than the old leverage, and it can not be larger than the maximum leverage.

During this function, the `oldLeverage` is calculated with the current values, and the `newLeverage` is calculated with the future values (expected values after the execution is completed).

```
uint256 currentBalance = jGMIndex.balanceOf(thisAddress) + newjGM;  
if (stableDebt > 0) {  
    uint256 jGMNeeded = _getjGMToPayback(stableDebt, _data.data.usdTotalValue);  
    underlying = currentBalance > jGMNeeded ? currentBalance - jGMNeeded : 0;  
} else {  
    underlying = currentBalance;  
}
```

`currentBalance`, `stableDebt` and `usdTotalValue` are three main values required while calculating the `newLeverage`.

The `currentBalance` is an updated value with the previewed `newjGM`. The `stableDebt` value is also updated before this calculation. However, the `_data.data.usdTotalValue` is still the current value, not the future USD value after this execution is completed.

The deposit of these borrowed funds will increase the `usdTotalVale`, which will affect the actual leverage, but it is not considered.

As a result of this, the calculated `newLeverage` during this function and the actual `leverage` after executing this function will be different and may result in reverting the transaction when the old/new [leverage validations

M-07 New Leverage Incorrectly Calculated During Leverage Up

CONTINUED

](https://github.com/GuardianAudits/jgm-pocs-1/blob/8e661971ca0a893d966192af907342d667e8e38d/src/leverage/LeverageStrategy.sol#L810C1-L816C10)are not passed.

RECOMMENDATION

Use the expected `usdTotalValue` while calculating the `newLeverage` for it to be precisely correct.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/43>

MEDIUM **M-08**

PARTIALLY RESOLVED

Value Extraction By Withdrawing After A Pay Back

CATEGORY Logical Error
LOCATION LeverageRouter.sol
REVIEW Main Review

DESCRIPTION

Users create withdrawals by interacting with the `LeverageRouter` contract. Users provide how many `jGM` shares to burn during withdrawal, and the `createWithdrawal` function calculates the corresponding `jGMIndex` amount, which will later be burned during GMX interaction and the user will get corresponding USDC.

`jGMIndex` amount is calculated with this formula:

```
if (supply > 0) {  
  _jGMIndex = _shares.mulDivDown(leverageStrategy.getUnderlyingjGM(data.data.usdTotalValue),  
    supply); // 18 decimals  
}
```

The result of the `getUnderlyingjGM` function directly impacts the amount user will get, and this function is directly affected by the `stableDebt`. Higher debt means less underlying tokens and vice versa.

The `payBack` function in the strategy contract decreases the `stableDebt`. Because of that, withdrawing an amount of `jGM` shares after a `payBack` results in more USDC compared to withdrawing before.

An attacker can abuse this if he knows when a pay back will be made and attackers can predict when a pay back will happen.

The protocol has a `keeperPayBack` function, which is called to increase stable token balance before paying the debt. Since this function withdraws stable tokens from the GMX, there will be a lag between `keeperPayBack` and the actual `payBack`.

- Attacker sees the `KeeperPayback` event is emitted during `withdrawalCallback`.
- Immediately deposits in the next block.
- The Jones operator calls the `payBack` and pays the debt.
- Attacker withdraws after pay back and profits.

M-08 Value Extraction By Withdrawing After A Pay Back

CONTINUED

RECOMMENDATION

The lag between `keeperPayBack` and the `payBack` gives chance to attackers to time their actions and deposit in between. Removing this lag would prevent the issue. Invoke the `IUnderlyingVault.payBack` function during the `withdrawalCallback` when the `action == 8` with appropriate parameters.

RESOLUTION

Status: Partially resolved.

<https://github.com/Jones-DAO/jones-gm/pull/45>

MEDIUM **M-09**

RESOLVED

Admin cannot change operationOngoing when stuck

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/master/src/gm/strategies/GMStrategy.sol#L26
REVIEW	Main Review

DESCRIPTION

When the protocol is put into a state in which `operationOngoing` is stuck to true users will not be able to interact with the protocol. Additionally there is no guarantee that retrying a deposit or withdraw will work. In which case users will not be able to interact with the protocol for a prolonged period of time.

RECOMMENDATION

Consider implementing an admin function that allows the protocol to change `operationOngoing` to true when the protocol is stuck and doing so is safe. However, it is important to note that the current deposit/withdraw would be lost as the following order will override the users callback data.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/53>

MEDIUM **M-10**

RESOLVED

Jones does not collect from jonesRate when protocolRate is zero

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/master/src/leverage/LeverageStrategy.sol#L373
REVIEW	Main Review

DESCRIPTION

When the `protocolRate` is zero the protocol will not be able to collect fees from `jonesRate` users withdraw. Because of this, the protocol will loose out on fee revenue.

The reason the fees are not collected when `protocolFee` is zero is because there is a check that requires both `protocolRate` and `jonesRate` to be non-zero. This is because the current calculation would set `jonesRetention` to zero if either of the variables were zero.

However, even if the protocol does not have a `protocolRate`, it should still be receiving the `jonesRate` portion.

RECOMMENDATION

To fix this add an else statement that calculates `jonesRetention` based on just `jonesRate`.

```
else { jonesRetention = _usdc.mulDivDown(jonesRate, BASIS_POINTS); }
```

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/37>

MEDIUM **M-11**

PARTIALLY RESOLVED

Users cannot borrow when stable vault is low on borrowable funds

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/master/src/leverage/LeverageStrategy.sol#L784
REVIEW	Main Review

DESCRIPTION

When a users deposit amount exceeds the amount of funds available in the stable vault deposits will not be possible. This situation can present itself when the stable vault is low on borrowable funds or when users make large deposits while `JGM` is below target leverage.

The issue with this is that there are still borrowable funds available and by reverting JGM will not be able to borrow when they should. And more importantly users would not be able to deposit into the protocol.

RECOMMENDATION

Instead of reverting when `stablesToBorrow` is greater than `availableForBorrowing`. Truncate the amount borrowed to what is available.

```
if (availableForBorrowing < stablesToBorrow) { stablesToBorrow = availableForBorrowing; }
```

RESOLUTION

Status: Partially resolved.

<https://github.com/Jones-DAO/jones-gm/pull/44>

MEDIUM **M-12**

ACKNOWLEDGED

Deterministic keeper actions can be gamed

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/master/src/gm/strategies/GMStrategy.sol#L643
REVIEW	Main Review

— DESCRIPTION

Because actions such as calling the `harvest` or `compound` functions are deterministic in the sense that a user knows at a certain threshold these function will be called. A user can extract value from other users by making timely deposits or withdrawals right before these functions are called.

This is because both of these function alter the share price. When the `harvest` function is called the share price will decrease as yield is paid to JUSDC. With the yield leaving the JGM protocol the value will decrease. A user can game this function by withdrawing funds right before the keeper calls `harvest` and then deposit right after. What this does is protect that user from the drop in share price by putting their burden on the other users.

When `compound` is called it will increase the share price by converting the unaccounted for incentive token into more GM tokens. As the amount of GM tokens increase so will the share value. A user can game this by depositing right before `compound` is called and withdraw right after. By doing this they are profiting off the compound function even though that yield should be going to the users who's deposits earned that yield.

In both cases users can game the system and extract value from other users.

— RECOMMENDATION

Ensure that keepers action are done at a time when performing such gaming strategies are not profitable. The thresholds for both `harvest` and `compound` should be low enough the fees outweigh the profit.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Strategy will payback users underlying when there is insufficient debt

CATEGORY Logical Error
LOCATION LeverageStrategy.sol: 161
REVIEW Main Review

DESCRIPTION

When a user withdraws from the stable vault, the `payback` function in the `LeverageStrategy` contract is called to decrease the `stableDebt` by the amount of USDC removed from the strategy. This process works correctly in most cases. However, if the amount being paid back exceeds the `stableDebt`, the `stableDebt` variable is simply set to 0. Consequently, the full USDC amount is transferred to the `stableVault` contract, but a portion of that USDC comes from users' collateral instead of the originally borrowed funds. This results in a loss of funds for all users involved.

RECOMMENDATION

Consider clearly documenting to users that in rare cases, the stable vault may utilize users' collateral when there's insufficient debt, particularly when a JUSDC user makes a withdrawal. This transparency helps users understand potential risks and ensures they are aware of how their funds may be utilized under specific circumstances.

RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

Misconfigured handler addresses will lead to failed callbacks

CATEGORY	Validation
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/master/src/gm/strategies/GMStrategy.sol#L908
REVIEW	Main Review

DESCRIPTION

In rare cases GMX can change the handler contracts if code needs to be added or removed. When this happens the handler address used by GMX will differ from the handler contract used by JGM. Because of this, deposit and withdraw creations will succeed, but the callback will fail leading to a temporary DoS until a keeper retries the callback.

In addition, in rare cases it is possible for GMX to have multiple handler contracts for a short period of time. When this happens if the handler contract that is not whitelisted performs the deposit execution the callback will fail as the `_onlyWithdrawHandlerOrKeeper` check will fail.

RECOMMENDATION

Consider validating the role of the `msg.sender` in the `RoleStore`, e.g. `RoleStore.hasRole(msg.sender, Role.CONTROLLER)`, this would check that the `msg.sender` is a valid handler.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/54>

LOW **L-03**

RESOLVED

Missing check if action does not equal 1 in Keeper PayBack

CATEGORY Validation

LOCATION <https://github.com/GuardianAudits/jgm-pocs-2/blob/8e661971ca0a893d966192af907342d667e8e38d/src/leverage/LeverageStrategy.sol#L696>

REVIEW Main Review

DESCRIPTION

Every function in the `LeverageStrategy` contract has a check at the beginning that there is no running action. Except for the `keeperPayBack` function, which is missing this check.

Under normal circumstances, this should not be a problem as the transaction should still revert because of the `operationOnGoing` check in the `GMStrategy` contract. However, if the `operationOnGoing` check does not work as expected the keeper bot could unintentionally overwrite another operation by calling `keeperPayBack`.

RECOMMENDATION

Add this check `if (action != 1) { revert InvalidAction(); }` to the `keeperPayBack` function.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/32>

LOW **L-04**

RESOLVED

Swapper Incorrect Allowance Handling

CATEGORY	Logical Error
LOCATION	UniswapV3Swapper.sol:138 UniswapV3Swapper.sol:154
REVIEW	Main Review

DESCRIPTION

The `UniswapV3Swapper` contract is used when any token swap is needed in the protocol. It incorrectly uses the `safeIncreaseAllowance` and `safeDecreaseAllowance` functions when working with tokens as if they would set the allowances, not increase/decrease it.

The swap, initiated through the `swap` function will always work, since on every call the allowance is increased with the given amount

```
IERC20(tokenIn).safeIncreaseAllowance(address(V3_ROUTER), amountIn);
```

but it will never decrement it, since it decreases it with 0, instead of setting it to 0.

```
IERC20(tokenIn).safeDecreaseAllowance(address(V3_ROUTER), 0);
```

This results in an continuous growth of allowance in the swapper contract.

RECOMMENDATION

Use the `[SafeERC20.forceApprove]`(<https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/token/ERC20/utils/SafeERC20.sol#L76>) function when setting and removing approval.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/25>

LOW **L-05**

RESOLVED

Protocol Withdraw Incentives Cannot Be Changed

CATEGORY Logical Error
LOCATION GMStrategy.sol:183-216 GMStrategy.sol:705-735
REVIEW Main Review

DESCRIPTION

On user withdraws, there are fees that are taken. These fees currently are only set on initialization and cannot be further changed.

Depending on economical factors, the fees might need to be changed.

RECOMMENDATION

Add a `updateIncentives` function in the `LeverageStrategy` contract where `protocolRate`, `jonesRate` and `incentiveReceiver` can be changed.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/48>

LOW **L-06**

RESOLVED

Keeper Pay Back Functionality Overwrites Current Action

CATEGORY	Logical Error
LOCATION	LeverageStrategy.sol:696-737
REVIEW	Main Review

DESCRIPTION

When a keeper calls the `keeperPayBack` function from the `LeverageStrategy` contract, if the required stable token amount is greater than available in the contract, a GMX withdraw is created.

The function however, incorrectly, does not check if there is already an action pending. As such, calling the `keeperPayBack` function during any already existing withdraw sets the action to 8 and when the first withdraw is finished, the action to be taken is that specific for the `keeperPayBack`, which is just emitting an event.

Also, the initial `keeperPayBack` withdraw would reach the withdraw callback with an action of 1 which will lead if on the code-path to the Liquidate behavior. Basically functionality is reversed.

RECOMMENDATION

In the `keeperPayBack` function, if the `expectedStables` are greater than 0, check that the current action is 1 (idle) otherwise revert the operation.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/32>

LOW **L-07**

RESOLVED

Fee Rates Can Dos Protocol

CATEGORY	Logical Error
LOCATION	LeverageRouter.sol:55 LeverageRouter.sol:158 LeverageStrategy.sol:117 LeverageStrategy.sol:118
REVIEW	Main Review

— DESCRIPTION

Throughout the system there are fees, both on deposits and on withdraws. The fees can be errantly set to over 100% BPS leading to full protocol DOS via underflow calculations. This applies to deposit rate fees, protocol date and Jones rate.

— RECOMMENDATION

Whenever setting the fee rates, validate that they are not over the equivalent of 100% in basis points.

— RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/47>

LOW **L-08**

RESOLVED

Deprecated Chainlink Staleness Check

CATEGORY	Logical Error
LOCATION	RebalanceStrategy.sol:316-370
REVIEW	Main Review

DESCRIPTION

Chainlink [has deprecated](<https://docs.chain.link/data-feeds/api-reference#latestrounddata>) the `answeredInRound` return variable for its `latestRoundData` price feed function and its usage is now discouraged.

This variable comes from a legacy version of Chainlink and it can be safely ignored.

RECOMMENDATION

Remove the check.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/46>

LOW **L-09**

RESOLVED

Possible Trapped ETH In Routers

CATEGORY Logical Error
LOCATION GMStrategy.sol:817-819
REVIEW Main Review

DESCRIPTION

The `LeverageRouter` and `GMRouter` contracts allow receiving of direct ETH through the existence of a payable `receive` function.

There is no functionality within the contract that uses any stored ETH. Also, there are no emergency withdraw functions in case ETH is wrongly sent to the contract.

RECOMMENDATION

Remove the `receive` function.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/41>

LOW **L-10**

RESOLVED

Leverage Down Reverts On Zero Underlying

CATEGORY Logical Error
LOCATION GMStrategy.sol:778
REVIEW Main Review

DESCRIPTION

When the system is over leveraged, the keeper logic determines the need to leverage down and calls the `leverageDown` function.

If this function is called when the system is over-leveraged to the point that the entire strategy `jGM` balance would be needed to pay back the debt (leaving 0 underlying) or when the underlying `jGM` balance is directly 0, then the function reverts unexpectedly with a divide by 0.

```
uint256 oldLeverage = ((currentBalance * BASIS_POINTS) / underlying);
```

Depending on how the implementation of the `leverageDown` function is intended, the above issue would either render it prematurely revert (instead of gracefully exit) or block such logic.

RECOMMENDATION

Either change the way the old leverage is calculated or move the underlying and stable debt validation before using it.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/36>

LOW **L-11**

RESOLVED

Allowances Should Be Cleared While Updating GMX Variables And Swappers

CATEGORY Logical Error
LOCATION LeverageStrategy.sol:850
REVIEW Main Review

DESCRIPTION

During the initialization of the `GMStrategy` contract, the swapper and the GMX router contracts were approved with infinite allowances to spend market tokens, long tokens and USDC.

The owner of the `GMStrategy` contract has the ability to update default swapper and GMX router. However, allowances of the previous contracts are not cleared during this update.

The owner must separately call the `approveAssets` function with a 0 value to clear these previous allowances. In case of the owner not clearing these approvals manually, these allowances can be abused.

RECOMMENDATION

Clear the previous contracts' allowances inside the update functions.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/35>

LOW **L-12**

RESOLVED

Leverage Strategy Should Not Be Updated During An Operation

CATEGORY Logical Error
LOCATION GMRouter.sol:57-58 LeverageRouter.sol:58-59
REVIEW Main Review

DESCRIPTION

When a withdraw/deposit is done to the `GMStrategy`, if there is a leverage strategy set, it will call specific callbacks on it.

The owner of the contract can change this contract address via the `updateLeverageStrategy` function.

If the address is changed during a withdraw/deposit operation (`operationOnGoing`) then users that may have initiated the action through the leverage contract will have their operations blocked and, in some cases, their leverage `jGM` not minted to them, causing loss of funds.

RECOMMENDATION

Do not allow the `updateLeverageStrategy` function to be called when there is an operation on-going.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/30>

LOW **L-13**

RESOLVED

Rebalance Purchases Can Be Done In Multiple Stages

CATEGORY Validation
LOCATION GMOracle.sol:194 jGMIndex.sol:188
REVIEW Main Review

DESCRIPTION

When a rebalance reaches stage 3: Withdrawal Finish, then the `purchaseRebalance` function can be called to initiate GMX deposits.

The `purchaseRebalance` function incorrectly allows calls from both stage 3 and stage 4: Start Purchase Stage. After the first call to stage 3, it takes the rebalance to the next stage, but any subsequent calls do not have stage 3 as a requirement, only stage 4.

Because of the above implementation, multiple calls to the function can be done.

RECOMMENDATION

If the described behavior is intended clearly document it. If only a call within stage 3 is allowed then modify the function to require it being called in stage 3.

There also should be a mechanism to be sure that the `USDC` that was retrieved from the withdraws is not lost within the contract, thus the contract balance of `USDC` must be 0 at the end of the call, otherwise revert.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/31>

LOW **L-14**

RESOLVED

Incentive Not Accounted For on Payback Stables Availability

CATEGORY	Validation
LOCATION	LeverageStrategy.sol:149
REVIEW	Main Review

— DESCRIPTION

When the leverage strategy needs to pay back a part of its debt to the stable vault, the `payBack` function is invoked by an operator. This function checks the stable token balance of the leverage strategy contract, checks if the balance is enough to pay the debt, and calls the `IUnderlyingVault.payBack`, which will transfer tokens.

In the `LeverageStrategy.payBack` function, the `IUnderlyingVault.payBack` is called with two parameters: `amount` and `data.gmxIncentive`. Here, the transferred token amount from `leverageStrategy` contract to stable vault is `amount - data.gmxIncentive`, not the whole `amount`.

The incentive amount is not transferred during this call. However, this incentive amount is not accounted while checking whether the contract has enough balance to pay the debt.

Example scenario:

- `amount` : 1000 USDC
- `strategyStables` : 950 USDC
- `data.gmxIncentive` : 100 USDC

A payback call in this scenario will revert with `NotEnoughStables` error due to [this check] (<https://github.com/GuardianAudits/jgm-pocs-1/blob/8e661971ca0a893d966192af907342d667e8e38d/src/leverage/LeverageStrategy.sol#L155>). However, amount to be transferred in this call is only 900 USDC, and the contract has enough balance to pay it.

— RECOMMENDATION

The incentive amount should be accounted alongside with the contract balance while checking the contract's ability to pay back its the debt.

L-14 Incentive Not Accounted For on Payback Stables Availability

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/Jones-DA0/jones-gm/pull/27>

LOW **L-15**

RESOLVED

Strategy Total Value Does Not Reflect Rebalancing Amounts

CATEGORY Logical Error
LOCATION LeverageStrategy.sol
REVIEW Main Review

DESCRIPTION

When a rebalance is initiated, the funds to be rebalanced are taken from the `GMStrategy` and sent to the `RebalanceStrategy` contract until the rebalance is done, before sending them back to the `GMStrategy`.

During this time, the GM token weight and total USD value of all GM tokens in the GM Strategy do not reflect reality. Any 3rd party integration relying on it will produce invalid results in their own logic.

RECOMMENDATION

Modify the `GMStrategy` contract so that during a rebalance, the funds that left the contract are still accounted when determining the token weight and total USD value (via the functions `tokenWeight` and `totalValue`).

Specifically in the `rebalanceInput` function, save the moved GM tokens in a mapping and reuse that mapping in the `tokenWeight` and `totalValue` functions by adding the saved values to the current strategy balance. When a rebalance is finished, clear the mapping.

RESOLUTION

Status: Resolved.

<https://github.com/Jones-DAO/jones-gm/pull/26>

CRITICAL **C-01**

ACKNOWLEDGED

User Withdrawals Are Accounted As Stable Debt Payment After A Failed Swap

CATEGORY Logical Error
LOCATION GMStrategy.sol:528-533
REVIEW Remediation Review

DESCRIPTION

When a user initiates a withdrawal, necessary funds are withdrawn from GMX to `GMStrategy` contract, the long token is swapped to USDC in the `afterWithdrawalExecution` callback and this value is stored. When all callbacks are done and the `callbackCounter` reaches to zero, withdrawn funds are transferred to the `LeverageStrategy` contract, the `LeverageStrategy.withdrawalCallback` is called, and finally, the `operationOnGoing` flag is set to false.

The swap is done in a try/catch block to prevent a DoS issue and `LeverageStrategy` action is set to 1 if the swap fails. However, setting the action to 1 will causes problems if the swap fails for any reason during a legitimate withdrawal by a regular user.

If the swap fails when the `callbackCounter` was not zero, the `LeverageStrategy` action will be set to 1, but `operationOnGoing` in the `GMStrategy` will still be true, which creates a mismatch between two contracts. After this point, when the remaining callbacks completed, if the resulted USDC amount is in the slippage range, `LeverageStrategy.withdrawalCallback` will be called but due to the action was already set to 1, the transaction flow will enter [this else block](<https://github.com/GuardianAudits/jgm-pocs-1/blob/8e661971ca0a893d966192af907342d667e8e38d/src/leverage/LeverageStrategy.sol#L405C11-L415C14>). User's withdrawal will be considered as a debt payment instead of a withdrawal and all the funds withdrawn will be sent to the stable vault. If the USDC amount is not in the range, the function will revert, withdrawal can't be completed and the `operationOnGoing` flag will not be set to false.

RECOMMENDATION

Because of the way GMX callback work in tandem with the `jGM` reverts should be avoided altogether but, since swap reverts cannot be avoided, a safe compromise is:

- in the catch branch of the try-catch
- emit an event (for off-chain logic)

C-01 User Withdrawals Are Accounted As Stable Debt Payment After A Failed Swap CONTINUED

- since the swap failed, the current logic is to revert but that simply blocks the protocol. Create a mapping with the non-swapped tokens amounts due to the user had allow the user, at will, to claim the tokens. An option is to also allow the keeper to call the function to send the failed-to-swap tokens to the user (or swap the before-hand), that can be done immediately. It is not a good design to simply revert, but our initial recommendation was faulty.
- also set in the try-catch a flag to skip the `_isInRange` check, Without this extra step, the transaction will likely revert since after the final GM swap, the check will most likely fail

— RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/79>

CRITICAL C-02

ACKNOWLEDGED

Attacker Can Repurpose Signatures To Damage Protocol

CATEGORY	Logical Error
LOCATION	GMRouter.sol:224-238
REVIEW	Remediation Review

DESCRIPTION

The entire original description is still valid, with the observation that having amounts in the signature is not suffice, since deposits use USDC that has 6 decimals and withdraws use GM tokens that have 18 decimals, adding the operation type is also needed.

An attacker can still have a signature generated for one operation (e.g. deposits) and initiate it for withdraws, closing all the `GMStrategy` GMX positions.

RECOMMENDATION

From the original recommendation, the following are still required:

- add operation type (use an Enum for operations) to `IGMRouter.Data` structure (requirement)
- in the `strategyDeposit` function from the `GMStrategy` contract, validate that the passed `USDC` `_amount` is equal to that used to buy all the GM tokens (extra validation)

RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/58>

CRITICAL C-02.2

ACKNOWLEDGED

Using L1 block number leads to DoS

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/audit-pr-merged/src/gm/strategies/GMStrategy.sol#L262
REVIEW	Remediation Review

DESCRIPTION

Currently `GMXData` uses a hash of `block.number` and `receiver`. However on Arbitrum `block.number` is the L1 `block.number`. Using this will result in the same vulnerability as there can be many Arbitrum blocks within a single Ethereum block.

RECOMMENDATION

Using the `ArbSys` precompile reference the L2 Arbitrum block number
`ArbSys(100).arbBlockNumber()`

RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/66>

CRITICAL C-03

ACKNOWLEDGED

USDC Withdraw Amount Not Updated After Protocol Trim

CATEGORY	Logical Error
LOCATION	GMStrategy.sol:998-1005
REVIEW	Remediation Review

DESCRIPTION

When a user initiates a withdraw the `_isInRange` function checks if the withdrawn `USDC` amount after the GMX execution is above the ideal amount, and if so, the surplus is sent to the protocol treasury.

This function, however, incorrectly does not return the trimmed value in that case, and returns the same value as passed, the `_actualUSDC`. Since the difference between the actual amount and idea was sent to the protocol treasury, after exiting the `_isInRange` call, the incorrect USDC amount is then sent to the user meaning that that different is again taken from the contract.

If the `GMStrategy` contract does not have enough stables to do the withdraw, then the `IERC20(USDC).transfer(data.user, data.usdc);` transfer will revert, leaving the `operationOnGoing` flag set to true and subsequent problems. An attacker can also time this abuse when the contract is low on stables effectively forcing a revert.

RECOMMENDATION

In the `_isInRange` function, after the USDC transfer to the treasury, set the `_actualUSDC` variable with the `_idealUSDC` variable.

RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/59>

HIGH **H-01**

ACKNOWLEDGED

GM Strategy Token Weight Is Errantly Determined

CATEGORY	Logical Error
LOCATION	GMViewer.sol:124-128
REVIEW	Remediation Review

DESCRIPTION

The GM index composition of the `jGM` vault is monitored to ensure it remains within a predetermined range. This is crucial to maintaining a healthy GM system and protocol robustness.

The target weight is read directly from the `GMViewer` contract using the `getGMWeight` function. This function incorrectly calculates the weights and returns them as 30 decimals values, instead of less than 12 decimals, [as they are intended](https://github.com/GuardianAudits/jgm-pocs-1/blob/audit-pr-merged/src/gm/strategies/GMStrategy.sol#L159-L161).

The error appears when multiplying the in-question Strategy GM token balance value, by omitting to divide by `1e18uint256` `currentValue = IERC20(gm.token).balanceOf(address(this)) * GMPrice(gm.oracle, gm.stalePeriod);`

Since this value is instrumental in maintaining the balance of the system and triggering a rebalance, it is crucial to have it correctly determined.

RECOMMENDATION

Divide the `currentValue` with `1e18` before dividing it to the total value.

RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/60>

HIGH **H-01.2**

ACKNOWLEDGED

Data can still be stale and impact the share price calculation

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/c40df2177c9cdcd8fbe3a9578efdd8b05db33c88/src/interfaces/gm/IGMStrategy.sol#L26
REVIEW	Remediation Review

DESCRIPTION

When a big price change happens between fetching the signed data param and executing the operation, the data param becomes stale, impacts the share price calculation, and leaves excess USDC in the `GMStrategy` contract.

The reason for this is, that the amount of GM tokens to buy is calculated with stale data from before the price change. This leaves all the excess USDC in the `GMStrategy` contract but still gives the user shares for the full USDC amount. This impacts the share price calculation of the next deposit as the amount of shares and GM tokens mismatch.

Here is an example of such a scenario:

- User fetches the signed data param (the amounts of GM tokens to buy is calculated on the current prices)
- The price of the GM tokens changes significantly
- User creates a deposit with the stale data param
- As the `usdTotalValue` is calculated on-chain the user receives shares based on the given USDC amount, but the GM token amounts are stale, and therefore less GM tokens are bought than could be with the given USDC amount
- The excess USDC is stuck inside the `GMStrategy` contract (emergency withdraw needed)
- The next deposit will receive excess shares as the amount of shares given to the user before does not match the amount of USD value added to the system and this messes with the share price calculation

RECOMMENDATION

This could be prevented by instead of passing in fixed amounts of GMX market tokens to buy, using the percentage values of how much of each market token should be bought with the given USDC amount. But as price data is still stale, reverts and big slippage could happen. Therefore calculating everything on-chain is the better solution.

H-01.2 Data can still be stale and impact the share price calculationCONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DA0/jones-gm/pull/67>

HIGH **H-02**

ACKNOWLEDGED

Wrong key used to check if execution is disabled.

CATEGORY	DoS
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/audit-pr-merged/src/leverage/LeverageRouter.sol#L160
REVIEW	Remediation Review

— DESCRIPTION

Currently createWithdraw uses `CREATE_WITHDRAWAL_FEATURE_DISABLED` but this only checks if the creation is enabled.

— RECOMMENDATION

Instead to prevent this attack use `EXECUTE_WITHDRAWAL_FEATURE_DISABLED`

— RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DA0/jones-gm/pull/68>

HIGH **H-03**

ACKNOWLEDGED

Attacker can donate USDC to halt rebalance.

CATEGORY	DoS
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/audit-pr-merged/src/gm/strategies/RebalanceStrategy.sol#L377
REVIEW	Remediation Review

— DESCRIPTION

Because the `purchaseRebalance` function checks that the balance of the contract is 0 and reverts if this is not the case an attacker can drip a dust amount of USDC during a rebalance to prevent the rebalance from continuing leaving the protocol in a stuck period where no one can use it.

— RECOMMENDATION

Remove the check:

```
if (IERC20(USDC).balanceOf(thisAddress) != 0)
```

Instead use the keepers to calculate the right amount to deposit based on the contracts balance.

— RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/69>

MEDIUM **M-01**

ACKNOWLEDGED

Incorrect Underlying jGM Calculations Due To Fees

CATEGORY Logical Error
LOCATION LeverageStrategy.sol:949-952
REVIEW Remediation Review

DESCRIPTION

Throughout the protocol, in the `LeverageStrategy` contract there is a `_getjGMToPayback` function which is used to estimate how much `jGM` tokens should be withdrawn in order to get the passed amount of USDC (debt).

```
function _getjGMToPayback(uint256 _debt, uint256 _totalUSDValue) private view returns (uint256) {  
    uint256 _jGMIndex = GMViewer.getPreviewDeposit(_debt, _totalUSDValue) / 2;  
    return (GMStrategy.addSlippage(_jGMIndex) + _jGMIndex).mulDivDown(10007, 10000);  
}
```

This function retrieves the required amount and increases half of it with the set GM Strategy slippage (defaulted to 0.5%). This is done to ensure that deposit with swaps that reach the slippage limit would still work since the withdraw amount has taken this into consideration. Adds the other half and then increases the overall amount by 0.07%

Adding the `0.07%` fee, specific for GMX [deposits or withdraws on one pair](https://gmx-docs.io/docs/trading/v2/#open--close-fees) is done incorrectly because the amount increase to equivalent fee taking is incorrectly added. Since out of the given amount a percentage % would be withdraw, we need have the amount increase as `amount = amount * BASIS / (BASIS - slippage)` so that when the fee is applied, the exact `amount` value is returned. `mulDivDown(10007, 10000)` incorrectly adds the 0.07%, resulting in less tokens.

Because of the way the current `_getjGMToPayback`, and implicit leverage is calculated, overall system appears very slightly more leveraged then it is in reality.

RECOMMENDATION

Add the amount increase to compensate GMX fee in the manner of the above described formula. Also consider setting the GMX fee through a contract setter then hard-coding it.

M-01 Incorrect Underlying jGM Calculations Due To Fees

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DA0/jones-gm/pull/61/files>

MEDIUM **M-01.2**

ACKNOWLEDGED

Admin cannot change rebalanceLock when stuck

CATEGORY DoS

LOCATION <https://github.com/GuardianAudits/jgm-pocs-2/blob/c40df2177c9cdcd8fbe3a9578efdd8b05db33c88/src/gm/strategies/RebalanceStrategy.sol#L138>

REVIEW Remediation Review

DESCRIPTION

In the `RebalanceStrategy` contract, the `rebalanceLock` can not be emergency updated to false. Therefore, if anything goes wrong during the rebalance flow the contract will be stuck in this state forever.

RECOMMENDATION

Consider implementing an emergency function to update the `rebalanceLock` as in the `GMStrategy` contract for `operationOnGoing`.

RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/70>

MEDIUM **M-02**

ACKNOWLEDGED

Deposits, withdraws And RetentionRefunds Are Not EIP712 Compliant

CATEGORY Logical Error
LOCATION LeverageStrategy.sol:978 GMRouter.sol:64
REVIEW Remediation Review

DESCRIPTION

When initiating a deposit, withdraw or validating the retention fund GMX incentive amount, callers provide a signature must that will be checked in the `_verifyParams` to be from the correct signer which will also validate the provided input data.

The contracts that use this functionality, both `LeverageStrategy` and `GMRouter` router inherit the `EIP712Upgradeable` contract but it do not properly handler the signature verification in relation to the EIP-712 standard.

In case of the `GMXRouter` the `dataTypeHash` is incorrect because it includes the `uint256 usdTotalValue` which has been removed from the `IGMRouter.Data` structure. Also, within the same type string, change the `IGMStrategy.GMData[] gmData` to `GMData[] gmData`.

On retention refunds, validation is done on a direct types, completely lacking a `[typeHash]` (<https://eips.ethereum.org/EIPS/eip-712#rationale-for-typehash>) or structure to hold the information.

```
address _signer = ECDSA.recover(_hashTypedDataV4(keccak256(abi.encode(nonce + 1, gmxIncentives))), signature);
```

RECOMMENDATION

Correctly implement the [EIP-712](<https://eips.ethereum.org/EIPS/eip-712>) verification by adding a structure for the GMX incentive and create the `typeHash` on it. Fix the underlying issues with the `dataTypeHash` as mentioned in the description.

RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/81>

MEDIUM **M-02.2**

ACKNOWLEDGED

Contract should be paused or removed right after unwind

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/audit-pr-merged/src/leverage/LeverageStrategy.sol#L437
REVIEW	Remediation Review

— DESCRIPTION

After an unwind happens there will be time between the unwind and the removal of the contract because this process is being done manually.

— RECOMMENDATION

To fix this consider having the contract be paused at the end of the unwind.

— RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DA0/jones-gm/pull/71>

MEDIUM **M-03.2**

ACKNOWLEDGED

Signatures cannot be created while operationOngoing is true

CATEGORY	DoS
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/c40df2177c9cdcd8f8e3a9578efdd8b05db33c88/src/gm/GMViewer.sol#L98
REVIEW	Remediation Review

— DESCRIPTION

Signatures cant be created when there is an ongoing operation. This means that the protocol will not be easily usable with naturally high traffic.

— RECOMMENDATION

Consider Implementing the pending USD fix so that signatures can be created while there is an ongoingOperation.

Otherwise clearly document that fetching signatures can fail with high traffic.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-04.2**

ACKNOWLEDGED

Minimum deposit amount are not checked on Admin orders

CATEGORY	DoS
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/audit-pr-merged/src/gm/strategies/RebalanceStrategy.sol#L150
REVIEW	Remediation Review

— DESCRIPTION

When actions such as `rebalance` `compound` and `harvest` take place there is no check that the amount being deposited/withdrawn is not less than the minimum amount that should be sent. Because of this the same issue can present itself in the admin flows.

— RECOMMENDATION

Either explicitly check that the amount is sufficient or perform the check using the keepers.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

jUSDC Payback Through keeperPayBack Does Not Use Strategy Balance

CATEGORY Logical Error
LOCATION LeverageStrategy.sol:421
REVIEW Remediation Review

— DESCRIPTION

When a jUSDC payback is initiated through the leverage strategy `keeperPayBack` function, only the amount resulted from withdrawing GM tokens is paid back, even though the `keeperPayBack` itself checks and initiates the withdraw only for the amount that it lacks.

Because of the above issue, coupled with the lack of a working direct `payback` function, the stable from the strategy can never be used in paying back. This severely impacts jUSDC functionality and leverage strategy debt management.

— RECOMMENDATION

In the `withdrawalCallback` on action 8, get the current contract USDC balance, subtract the `gmxIncentives` from it, and pay it back. It already includes the `_usdc` amount, as it was transferred to the contract when the GMX callback was executed.

— RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/80>

MEDIUM **M-04**

ACKNOWLEDGED

jUSDC Cannot Enforce Payback

CATEGORY Logical Error
LOCATION LeverageStrategy.sol:157-164
REVIEW Remediation Review

— DESCRIPTION

The jUSDC vault has a functionality of enforcing the strategies that borrowed from it to payback a part of the debt. This is done through the `payBack` callback implemented in each strategy.

The `LeverageStrategy` contract has the `payBack` function reduced to a `return 0;` and all payback functionality was migrated through the `keeperPayBack` function.

This presents a severe issue within the Jones ecosystem as enforcing payback must also be allowed through the normal, jUSDC channel.

— RECOMMENDATION

Add the payback code back to the function.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

Hardcoded slippage

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/audit-pr-merged/src/leverage/LeverageStrategy.sol#L951
REVIEW	Remediation Review

DESCRIPTION

Currently the `_getjGMToPayback` hardcoded the fee amount as 0.07%. GMX has the ability to change the fee and when that happens the `_getjGMToPayback` will be wrong leading to a misleading payback amount.

RECOMMENDATION

Don't hardcode the fee so that admin can change it.

RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DA0/jones-gm/pull/61/files>

MEDIUM **M-06**

ACKNOWLEDGED

Pool is not always 50/50 leading to bad calculations.

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/audit-pr-merged/src/leverage/LeverageStrategy.sol#L950
REVIEW	Remediation Review

— DESCRIPTION

Currently the `_jGMIndex` is calculated as follows `uint256 _jGMIndex = GMViewer.getPreviewDeposit(_debt, _totalUSDValue) / 2;`

The issue here is that pools are often not 50/50. Because of this the calculation will either overestimate or underestimate the amount needed to payback.

— RECOMMENDATION

Consider using the actual pool ratios instead of dividing by two. Otherwise monitor the pool ratio to ensure the `_getjGMToPayback` function is not skewed.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

newTotalUSD is based on borrowing non truncated amount

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/audit-pr-merged/src/leverage/LeverageStrategy.sol#L825
REVIEW	Remediation Review

DESCRIPTION

Currently the `newTotalUSD` is based on the amount that is expected to be leveraged up/down. However because the amount can be truncated if there is not enough to borrow, the `newTotalUSD` will not be accurate and overestimate the amount actual amount.

RECOMMENDATION

Calculate `newTotalUSD` based on the actual amount being borrowed not the expected (pre-truncated) amount.

RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DA0/jones-gm/pull/72>

MEDIUM **M-08**

ACKNOWLEDGED

Execution feature disabled checks not implemented in admin flows

CATEGORY Validation

LOCATION <https://github.com/GuardianAudits/jgm-pocs-2/blob/c40df2177c9cdcd8f8e3a9578efdd8b05db33c88/src/leverage/LeverageRouter.sol#L106-L108>

REVIEW Remediation Review

DESCRIPTION

The GMX Keeper has the authority to disable specific features when necessary, such as executing deposits and withdrawals. If these features are disabled, attempts by keepers to execute orders will result in reverts, causing the orders to be canceled. Subsequently, if the JGM keeper tries to recreate the deposit with altered parameters for a successful order, it will also fail due to the disabled feature. This situation presents a critical issue as JGM users will be unable to interact with the protocol entirely, even if only one of the deposits or withdrawals is disabled.

Therefore the protocol implemented checks if the given action (deposit, or withdraw) is disabled on GMX. However these checks are only implemented in the user flows, any keeper/admin function does not have these checks implemented. Therefore the keeper bot could automatically execute a function like for example `leverageUp` or `leverageDown` as a threshold is reached and DoS the whole system by doing so.

RECOMMENDATION

Implement the same checks in the keeper/admin flows as well. By putting the check inside the `GMRouter` contract instead of the `LeverageRouter`, the checks can already capture most of the flows (only compound and rebalance missing).

RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DA0/jones-gm/pull/73>

LOW **L-01**

ACKNOWLEDGED

Keeper Can Be Made To Lose Gas On Failing Pending Deposits

CATEGORY Logical Error
LOCATION GMStrategy.sol:354-366
REVIEW Remediation Review

DESCRIPTION

The `afterDepositCancellation` function trigger the bot to retry a cancelled order by calling either `executeSingleDeposit` based on the emitted data from the cancellation. However, any user can set the `callbackContract` to the `GMStrategy` contract on their own malicious contract.

This allows an attacker to orchestrate a guaranteed-to-fail deposit with the `GMStrategy` as the callback contract. Subsequently, when the cancellation callback is invoked, the keeper will retry the order with the attacker key and revert with `Unauthorized` due to the key check.

This revert causes slight gas costs for the keeper (if he does not pre-validate if the function will revert).

RECOMMENDATION

Add the check that `keys[key]` is different from 0 in the `afterDepositCancellation` function, similar to how it is done in the `afterWithdrawalCancellation` function.

RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/62>

LOW **L-01.2**

ACKNOWLEDGED

IncentiveReceiver receives less funds in KeeperPayback flow when USDC depegs

CATEGORY Validation

LOCATION <https://github.com/GuardianAudits/jgm-pocs-2/blob/c40df2177c9cdcd8f8e3a9578efdd8b05db33c88/src/leverage/LeverageStrategy.sol#L419>

REVIEW Remediation Review

DESCRIPTION

At the end of the `KeeperPayback` flow the `gmxCoincentives` are calculated and sent to the `incentiveReceiver`. To calculate the `gmxCoincentives` the ETH/USD oracle is used and the given amount in USDC is transferred.

If USDC depegs from USD the `incentiveReceiver` will receive to less tokens measured in USD.

RECOMMENDATION

Consider converting the USD value to USDC.

RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/74>

LOW **L-02**

ACKNOWLEDGED

Potential Native ETH Trapped

CATEGORY	Logical Error
LOCATION	LeverageRouter.sol:117 LeverageRouter.sol:172
REVIEW	Remediation Review

— DESCRIPTION

Whenever a user deposits or withdraws from the protocol they need to pass native ETH that will be used as an execution fee for the GMX keepers.

This amount is only checked to be enough, but extra ETH, if sent, is considered donated or remains blocked in the strategy contract until retrieved via emergency withdraw.

```
msg.value < data.data.executionAmount
```

— RECOMMENDATION

Change so that the `msg.value` must be equal to `executionAmount` on checks.

— RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/63>

LOW **L-02.2**

ACKNOWLEDGED

Allowances not cleared while updating GMX variables and swappers in the RebalanceStrategy

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/c40df2177c9cdcd8fbe3a9578efdd8b05db33c88/src/gm/strategies/GMStrategy.sol#L779-L789
REVIEW	Remediation Review

DESCRIPTION

During the initialization of the `GMStrategy` and `RebalanceStrategy` contracts, the swapper and the GMX router contracts were approved with infinite allowances to spend market tokens, long tokens, and USDC.

The owner of the `GMStrategy` contract can update the swappers and GMX router. However, allowances of the previous contracts are only cleared for the GMX router in the `GMStrategy` contract. The allowances for the swappers in both the `GMStrategy` and `RebalanceStrategy` contracts are not cleared and the GMX router allowance in the `RebalanceStrategy` contract is not cleared.

RECOMMENDATION

Clear the previous contract allowances inside the update functions.

RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/75>

LOW **L-03.2**

ACKNOWLEDGED

Sum of fees can exceed 100% and prevent execution

CATEGORY	Validation
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/audit-pr-merged/src/leverage/LeverageStrategy.sol#L587
REVIEW	Remediation Review

— DESCRIPTION

The protocolFee + jonesRate can exceed 100% when this happens withdraws will revert as the amount being taken from fees will be greater than the amount that exist.

— RECOMMENDATION

When setting fees ensure that the sum is does not exceed 100%

— RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/76>

LOW **L-03**

ACKNOWLEDGED

Trapped ETH On keeperPayBack Call

CATEGORY	Logical Error
LOCATION	LeverageStrategy.sol:719-764
REVIEW	Remediation Review

DESCRIPTION

When a keeper calls the `keeperPayBack` function from the `LeverageStrategy` contract, if the contract does not have enough stables, a GMX withdraw routine will be initiated. Within that GMX withdraw routine, several GM tokens may be withdrawn. For each GM token, the keeper must provide enough native ETH so that GMX's own keeper finalizes the withdraw.

With regards to the way native ETH is handled in this function, there is an issue that if the function is called but, by coincidence there are enough stable tokens then the function secedes any ETH sent to it by the Keeper remains blocked, until it will be extracted via the `emergencyWithdraw` function.

RECOMMENDATION

If the functions determines that there are enough stables in it (`expectedStables == 0`) then either revert or send the native ETH back to the caller.

RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/64>

LOW **L-04**

ACKNOWLEDGED

rebalanceAmounts not implemented in getGMWeight function

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/c40df2177c9cdcd8fbe3a9578efdd8b05db33c88/src/gm/GMViewer.sol#L124-L128
REVIEW	Remediation Review

— DESCRIPTION

When a rebalance is initiated, the funds to be rebalanced are taken from the `GMStrategy` and sent to the `RebalanceStrategy` contract until the rebalance is done, before sending them back to the `GMStrategy`.

During this time, the GM token weight and total USD value of all GM tokens in the GM Strategy do not reflect reality. Any 3rd party integration relying on it will produce invalid results in their own logic.

To fix this issue `rebalanceAmounts` were added into a mapping and applied to the `getTotalValue`, but they are still missing in the `getGMWeight` function.

— RECOMMENDATION

Implement the `rebalanceAmounts` into the `getGMWeight` function calculations.

— RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/77>

LOW **L-05**

ACKNOWLEDGED

Attacker can still DoS even with minimum amounts being applied

CATEGORY Logical Error
LOCATION Global
REVIEW Remediation Review

— DESCRIPTION

Even with a minimum users can still make deposits and withdrawals at a constant rate DOSing the protocol. This only makes the attack a little more expensive.

— RECOMMENDATION

Consider removing a global lock however this would become a large change to the whole system. If no action is taken then this should be monitored and if this were to happen temporarily changing the fee % would help make this attack not feasible.

— RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/78>

LOW **L-07**

ACKNOWLEDGED

Funds still trapped from overspending as well as refund from GMX

CATEGORY	DoS
LOCATION	https://github.com/GuardianAudits/jgm-pocs-2/blob/audit-pr-merged/src/gm/strategies/GMStrategy.sol#L317 https://github.com/GuardianAudits/jgm-pocs-2/blob/audit-pr-merged/src/leverage/LeverageRouter.sol#L172
REVIEW	Remediation Review

— DESCRIPTION

Users can still send excess `msg.value` and have those funds locked in the protocol.

Users also do not get the excess execution fee returned to them, resulting in excess WETH locked in the GMStrategy.

— RECOMMENDATION

Require that `msg.value` is `==` to the execution fee. Not `msg.value` is `>=` to the execution fee.

Consider refunding the excess WETH that is returned after a deposit/withdrawal is executed. Otherwise document that excess execution fee will NOT be refunded.

— RESOLUTION

Status: Acknowledged.

<https://github.com/Jones-DAO/jones-gm/pull/63>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.